

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 1

з дисципліни «Системне програмування вбудованих систем»
Налаштування середовища розробки та запуск ParcelLog на STM32 у Wokwi

Виконав: студент групи ІО-33
Чекун Микола Олексійович
Перевірив: Гордієнко Нікіта Юрійович

Київ 2026

Мета роботи

Налаштувати відтворювану збірку прошивки для STM32C031C6, підготувати конфігурацію периферії STM32 HAL, запустити прошивку у Wokwi та отримати повідомлення через USART2. Для подальшого проекту визначити датчики, їхні інтерфейси та правила перевірки.

Тема та склад прототипу

ParcelLog реєструє стан однієї посилки під час перевезення та зберігання. MPU6050 виявляє удар або нахил, фоторезистор відстежує зміну освітлення всередині упаковки, PIR повідомляє про рух поруч лише в режимі зберігання. Жоден із цих сигналів сам по собі не встановлює, хто відкрив посилку або чи відбулося відкриття.

Компонент	Інтерфейс	Підключення та призначення
MPU6050	I ² C через GPIO HAL	PB8 SCL, PB9 SDA; події shock і tilt
Фоторезистор	ADC1	PA0 / ADC1_IN0; подія light_change
PIR	GPIO	PB1; nearby_motion лише у storage
Перемикач	GPIO	PB0; вибір transit або storage
Журнал	USART2	PA2 TX, PA3 RX; 115200 бод
Індикатор	GPIO	PA5; світлодіод роботи

Проект використовує віртуальну NUCLEO-C031C6. Фізичної плати й вимірювань на реальному обладнанні немає.

Конфігурація STM32

У firmware/firmware.ioc обрано STM32C031C6, 12-бітний ADC1 на PA0, USART2 у асинхронному режимі, входи PB0 і PB1 та лінії PB8/PB9. Частота SYSCLK, HCLK і APB дорівнює 48 МГц. У робочому коді обмін I²C з MPU6050 виконується програмно через GPIO HAL на PB8/PB9, оскільки апаратний I²C1 у цій конфігурації Wokwi не давав коректних відповідей датчика. Файл .ioc збережено в репозиторії для повторення налаштування.

Хід виконання

Спочатку створено Docker-образ з arm-none-eabi-gcc і налаштовано Makefile. Команди make clean та make build виконуються з кореня проекту. Після цього додано конфігурацію STM32, схему diagram.json і wokwi.toml, а також UART-журнал. Вимоги та спосіб перевірки записані в docs/PRD.md.

```
chekunm@Ubuntu24:~/ParcelLog$ make clean
make -C firmware clean
make[1]: Entering directory '/home/chekunm/ParcelLog/firmware'
rm -fR build
make[1]: Leaving directory '/home/chekunm/ParcelLog/firmware'
chekunm@Ubuntu24:~/ParcelLog$
```

Рисунок 1. Очищення результатів попередньої збірки на Ubuntu VM.

```

chekunm@Ubuntu24:~/ParcelLog$ make build
docker build -t stm32c0-wokwi-build:local docker
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.

                Install the buildx component to build images with BuildKit:
                https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:22.04
--> b8b6ee6aa931
Step 2/4 : ARG DEBIAN_FRONTEND=noninteractive
--> Using cache
--> 2a3bade6f42e
Step 3/4 : RUN apt-get update && apt-get install -y --no-install-recommends
make gcc-arm-none-eabi binutils-arm-none-eabi libnewlib-arm-none-eabi
ca-certificates && rm -rf /var/lib/apt/lists/*
--> Using cache
--> a03a80407cf5

```

Рисунок 2. Запуск збірки через Docker на Ubuntu VM. Попередження про legacy builder не зупинило збірку.

```

arm-none-eabi-gcc build/main.o build/mpu6050.o build/parcel_rules.o build/stm32c0xx_it.o build/stm32c0xx_hal_msp.o build/syscalls.o build/system.o build/stm32c0xx_hal.o build/stm32c0xx_hal_adc.o build/stm32c0xx_hal_adc_ex.o build/stm32c0xx_hal_cortex.o build/stm32c0xx_hal_rcc.o build/stm32c0xx_hal_rcc_ex.o build/stm32c0xx_hal_flash.o build/stm32c0xx_hal_flash_ex.o build/stm32c0xx_hal_gpio.o build/stm32c0xx_hal_dma.o build/stm32c0xx_hal_dma_ex.o build/stm32c0xx_hal_pwr.o build/stm32c0xx_hal_pwr_ex.o build/stm32c0xx_hal_exti.o build/stm32c0xx_hal_i2c.o build/stm32c0xx_hal_i2c_ex.o build/stm32c0xx_hal_uart.o build/stm32c0xx_hal_uart_ex.o build/system_stm32c0xx.o build/startup_stm32c031xx.o -mcpu=cortex-m0plus -mthumb -specs=nano.specs -TSTM32C031C6Tx_FLASH.ld -lc -lm -lnosys -Wl,-Map=build/firmware.map,--cref -Wl,--gc-sections -o build/firmware.elf
arm-none-eabi-size build/firmware.elf
text      data      bss      dec      hex filename
19180     120     1904    21204    52d4 build/firmware.elf
arm-none-eabi-objcopy -O ihex build/firmware.elf build/firmware.hex
arm-none-eabi-objcopy -O binary -S build/firmware.elf build/firmware.bin
make: Leaving directory '/workspace/firmware'
chekunm@Ubuntu24:~/ParcelLog$

```

Рисунок 3. Успішне створення ELF, HEX і BIN на Ubuntu VM, знімок від 26.09.2026.

На рисунку 3 зафіксована попередня ревізія прошивки з text 19 180 байт. Після виправлення обміну з MPU6050 поточна локальна збірка має text 18 456 байт, data 120 байт і bss 1 904 байти. Це різні ревізії, тому розміри не збігаються.

Робота у Wokwi

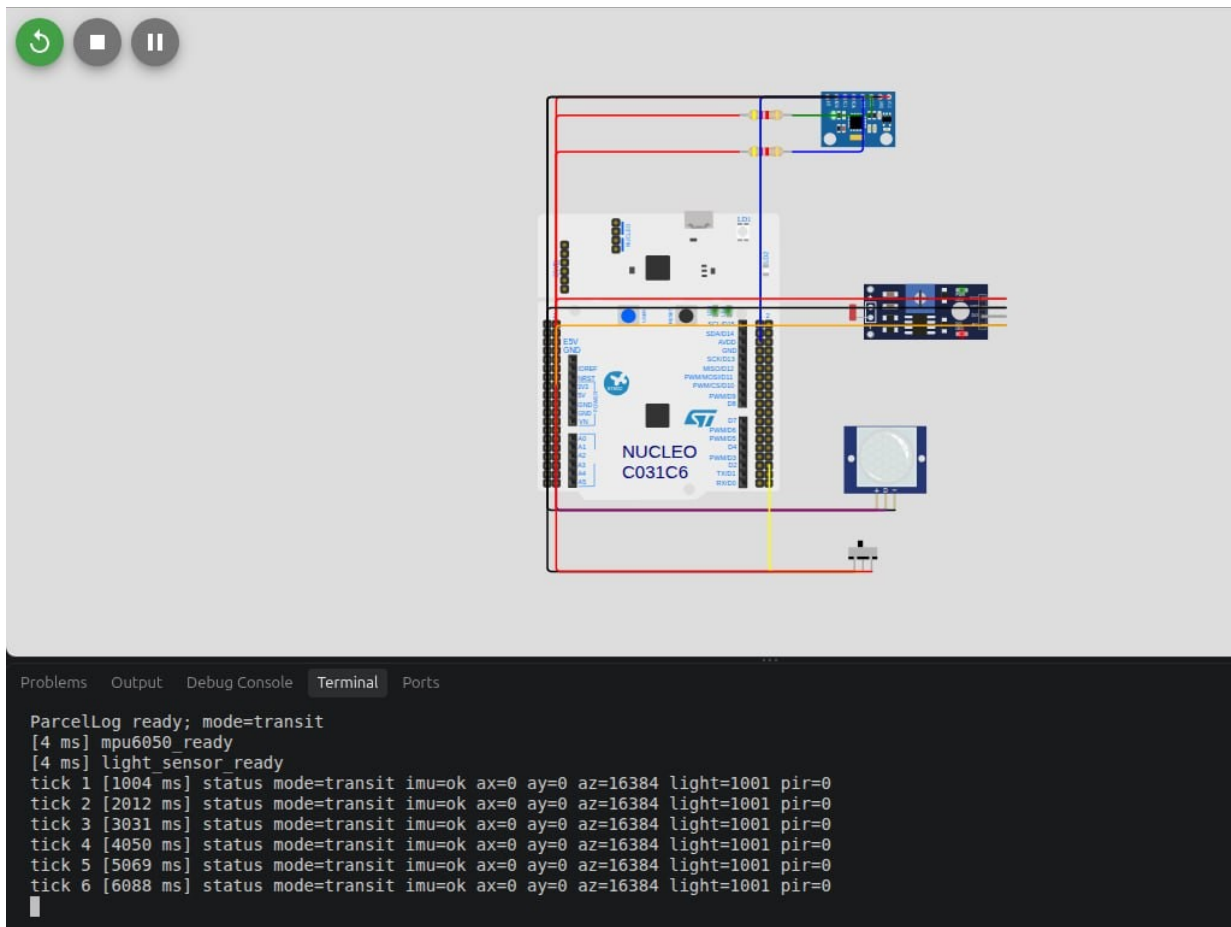


Рисунок 4. Схема Wokwi та UART-журнал: `mpu6050_ready`, `light_sensor_ready`, `imu=ok` і щосекундні повідомлення `tick`.

Після запуску симулятор повідомляє `mpu6050_ready` та `light_sensor_ready`. У стані `transit` датчик MPU6050 відповідає, а нерухомий модуль показує `az=16384`, що відповідає приблизно 1 g для діапазону ± 2 g. Журнал показує час від запуску, а не календарний час. Після перезапуску ці повідомлення не зберігаються.

Перевірка та труднощі

Команда `make test` перевіряє правила подій на комп'ютері й завершується рядком `parcel_rules: ok`. Скріншот Wokwi підтверджує запуск і читання MPU6050 у спокої. Окремі сценарії `shock`, `tilt`, `light_change` та `nearby_motion` цими чотирма знімками не підтверджені.

Некоректні ідентифікатори контактів Wokwi

Схема не відповідала назвам контактів моделі NUCLEO-C031C6. Причиною були некоректні ID pin у `diagram.json`. Їх замінено на ID, які використовує Wokwi для цієї плати, у коміті 9846d4e.

MPU6050 був недоступний у симуляції

У UART з'явилися `mpu6050_unavailable` та `imu=error`. Апаратний I²C1 на цій віртуальній платі не давав потрібної відповіді. Обмін перенесено на програмний I²C через GPIO HAL із підтягуванням ліній PB8/PB9. Після виправлення у коміті 4fafdbb з'явилися `mpu6050_ready` та `imu=ok`, як на рисунку 4.

У Docker з'являється попередження про застарілий `legacy builder`. Воно не є помилкою компіляції: після нього створено всі три файли прошивки. Для відтворення збірки цей факт слід відрізнити від помилки `make`.

Висновок

Підготовлено збірку STM32 HAL, збережено конфігурацію периферії, запущено ParcelLog у Wokwi й отримано UART-журнал із часовими мітками. Після виправлення схеми та I²C датчик MPU6050 відповідає. Для наступної перевірки потрібно окремо показати події удару, нахилу, зміни світла та руху PIR у режимі `storage`. Поточний журнал не є постійним сховищем і не дає підстав встановлювати особу, яка взаємодіяла з посилкою.

Матеріали та посилання

1. Репозиторій ParcelLog. <https://github.com/chekunm/ParcelLog>
2. Вимоги ParcelLog. <https://github.com/chekunm/ParcelLog/blob/main/docs/PRD.md>
3. Wokwi NUCLEO-C031C6. <https://docs.wokwi.com/parts/board-st-nucleo-c031c6>
4. Wokwi MPU6050. <https://docs.wokwi.com/parts/wokwi-mpu6050>
5. Wokwi photoresistor sensor. <https://docs.wokwi.com/parts/wokwi-photoresistor-sensor>
6. Wokwi PIR motion sensor. <https://docs.wokwi.com/parts/wokwi-pir-motion-sensor>
7. TDK MPU-6050. https://product.tdk.com/en/search/sensor/motion-inertial/imu/info?part_no=MPU-6050
8. Проблема I²C NUCLEO-C031C6 у Wokwi. <https://github.com/wokwi/wokwi-features/issues/1001>
9. Коміт зі схемою та першою прошивкою. <https://github.com/chekunm/ParcelLog/commit/89470dc>
10. Коміт із виправленням контактів Wokwi. <https://github.com/chekunm/ParcelLog/commit/9846d4e>
11. Коміт із виправленням MPU6050. <https://github.com/chekunm/ParcelLog/commit/4fafdbb>